Software Design Patterns (SDP) — Assignment 4

Facade Over a Three-Stage Chain, With Persistence and a Live Demo

Student: Nodir Mukhammedov

Group: SE-2536

Assigned Domain: Variant 8 — Flight Reservation

GitHub Repository: https://github.com/Eqziz/SDP-assignment-4.git

1. Executive Summary & System Architecture

Modern enterprise systems frequently require coordinating multiple autonomous subsystems behind a unified business transaction. In this project, a complete, highly durable **Flight Reservation System** is designed and implemented using the **Facade Pattern**.

The system coordinates three autonomous subsystems in a sequential chain:

- **Step A (Intake / Seat Reservation):** Validates seat maps, computes class/location pricing, and establishes a durable seat hold (`HELD`).
- **Step B (Processing / Execution):** Enforces baggage policies, assesses excess weight fees, generates official IATA ticket serial numbers, and issues e-tickets (`ISSUED`).
- **Step C (Finalization / Confirmation):** Validates passenger loyalty eligibility and catering meal restrictions, credits frequent-flyer miles, and finalizes the reservation (`SETTLED`).

The core architectural challenge is handling partial failures: real chains break midway after earlier steps have already written committed state to durable disk storage. The Facade orchestrates the execution, logs progress into a persistent transaction journal, detects downstream failures, and reliably unwinds the chain across two distinct compensation depths.

> **Mandatory Constraints Fully Satisfied:** All three subsystems are completely decoupled (zero cross-subsystem imports), state transitions are strictly self-enforced by domain entities, state persists durably across JVM restarts using JSON files, unit test coverage exceeds 94%, and all source code is strictly written without comments.

2. Independent Subsystems Breakdown

Each subsystem functions as a self-contained domain module consisting of 4 to 5 classes. Each subsystem features dedicated pure business logic classes separate from its entity, enforces its own state transition rules, and communicates directly with its own embedded JSON file store. No subsystem imports or knows about any other subsystem.

Subsystem	Class Name	Type / Responsibility	Durable File Gateway
Subsystem A (Seat Reservation)	`SeatReservation`	Domain Entity (States: `HELD`, `CONFIRMED`, `RELEASED`)	`SeatReservationRepository` (`data/seat_reservations.json`)
	`FareCalculator`	Pure Business Logic (Cabin rates, exit rows, taxes)	
	`SeatMapValidator`	Pure Business Logic (Aircraft seat grid & conflict checks)	
	`SeatReservationStatus`	State Enum with legal transition enforcement	
Subsystem B (Ticket & Baggage)	`TicketIssuance`	Domain Entity (States: `ISSUED`, `COMPLETED`, `VOIDED`)	`TicketIssuanceRepository` (`data/ticket_issuances.json`)
	`BaggageRulesEngine`	Pure Business Logic (20kg allowance, 32kg max safety limit)	
	`TicketNumberGenerator`	Pure Business Logic (IATA standard format: `ETKT-2026-XXXXXX`)	
	`TicketStatus`	State Enum with legal transition enforcement	
Subsystem C (Loyalty & Catering)	`LoyaltyAccountRecord`	Domain Entity (States: `ACCRUED`, `SETTLED`, `REVOKED`)	`LoyaltyRecordRepository` (`data/loyalty_records.json`)
	`LoyaltyPointsCalculator`	Pure Business Logic (Tier multipliers: Standard to Platinum)	
	`MealPreferenceValidator`	Pure Business Logic (Catering policy & account suspension checks)	
	`LoyaltyStatus`	State Enum with legal transition enforcement	

3. The Dual Facade Architecture

Two distinct facades encapsulate the trio of subsystems, catering to different client personas and operational responsibilities:

1. FlightBookingFacade (Primary Client Workflow)

Provides the standard client entry point: `BookingResult bookFlight(BookingRequest request)`. Rather than acting as a superficial delegate, this facade contains real sequencing and transactional decision logic:

- **Input & Pre-Commit Validation:** Validates passenger credentials and ensures the requested seat is structurally valid and unoccupied before writing any records to disk.
- **Step A (Seat Hold):** Computes cabin pricing, generates reservation identifiers, transitions seat entity to `HELD`, and commits to disk.
- **Step B (Ticket Issuance):** Evaluates baggage limits. If valid, generates an e-ticket and commits to disk. If baggage exceeds 32.0 kg, intercepts `ExcessBaggageException` and executes **Compensation Depth 1**.
- **Step C (Loyalty Confirmation):** Validates meal preferences and checks for suspended accounts. If verified, computes points, commits loyalty records, and promotes all three subsystem entities to their final settled states (`CONFIRMED`, `COMPLETED`, `SETTLED`). If an account is suspended, intercepts `LoyaltyProcessingException` and executes **Compensation Depth 2**.

2. FlightOperationsFacade (Administrative & Recovery Workflow)

Provides operations, manual intervention, and audit capabilities:

- `inspectBooking(bookingReference):` Reads correlated records from all three subsystem repositories and the journal to verify cross-system atomic consistency.
- `reconcilePendingTransactions():` Post-crash recovery engine. Reads the journal from disk after an unexpected JVM termination, detects unfinished/interrupted chains, and safely executes rollbacks.
- `manualRollback(bookingReference):` Allows authorized administrators to manually unwind confirmed or partially completed bookings.
- `resetAllStores():` Completely purges all disk files for repeatable demonstrations.

4. Durable Persistence & Crash Recovery

Persistence is real and durable: all subsystem repositories serialize domain entities to JSON files via Jackson Databind and JSR-310 datetime modules. To survive application restarts mid-chain without relying on volatile in-memory state, the chain's execution progress itself is durably journaled:

> **Persisted Chain Lifecycle Stages:**
> `INITIALIZED` → `STEP_A_SEAT_HELD` → `STEP_B_TICKET_ISSUED` → `COMPLETED`
> *On Step B Failure:* `COMPENSATING_AFTER_B_FAILURE` → `COMPENSATED_A_ONLY`
> *On Step C Failure:* `COMPENSATING_AFTER_C_FAILURE` → `COMPENSATED_FULL`

When the system restarts, `FlightOperationsFacade.reconcilePendingTransactions()` reads `data/booking_journal.json` directly from disk. Any transactions found in an intermediate state (e.g. `STEP_A_SEAT_HELD` or `STEP_B_TICKET_ISSUED`) are automatically identified, and their persisted entities in Subsystems A and B are unwound to clean terminal states (`RELEASED` and `VOIDED`).

5. Cross-Subsystem Consistency & Two-Depth Compensation

The system models two realistic, domain-specific failure scenarios that mandate rollback:

Compensation Depth 1: Step B Fails After Step A Committed

Domain Trigger: Passenger requests a checked bag weighing 40.0 kg. The airline's maximum single-bag safety limit is 32.0 kg. Step A has already committed seat hold `14B` as `HELD`. Step B's `BaggageRulesEngine` rejects the baggage with `ExcessBaggageException`.

Unwind Sequence: The facade catches the exception, updates the transaction stage to `COMPENSATING_AFTER_B_FAILURE`, and compensates Subsystem A alone: `SeatReservation.transitionTo(RELEASED)` is invoked and written to disk. Journal advances to `COMPENSATED_A_ONLY`.

Compensation Depth 2: Step C Fails After Steps A and B Committed

Domain Trigger: Passenger holds seat `16C` (`HELD`) and ticket `ETKT-2026-100001` is successfully issued (`ISSUED`). However, during Step C, the passenger's member ID is flagged as `SUSPENDED_USER_88`. `MealPreferenceValidator` rejects the request with `LoyaltyProcessingException`.

Two-Step Unwind Sequence: The facade updates the journal to `COMPENSATING_AFTER_C_FAILURE` and unwinds the chain in reverse order:

1. **Step 1:** Compensates Subsystem B → `TicketIssuance.transitionTo(VOIDED)` and updates disk.
2. **Step 2:** Compensates Subsystem A → `SeatReservation.transitionTo(RELEASED)` and updates disk.
3. Journal advances to `COMPENSATED_FULL`.

Pre-Commit Validation Failure at Step A

If seat format is invalid (e.g., `99Z`) or seat is already occupied on that flight, `SeatMapValidator` throws `SeatUnavailableException` before any records are committed to disk, keeping storage pristine.

6. Principle of Least Knowledge (Law of Demeter)

The Principle of Least Knowledge dictates that a software component should only interact with its immediate friends and avoid traversing complex subsystem graphs.

Before Facade: Deep Chains & Massive Coupling

Without the facade, a client booking a flight would need direct coupling to at least 10 classes across three disparate subsystems. The client would be forced to coordinate sequencing, track multiple repositories, and manually execute reverse rollbacks when downstream checks fail:

```
SeatMapValidator validator = new SeatMapValidator();
FareCalculator fareCalc = new FareCalculator();
SeatReservationRepository seatRepo = new SeatReservationRepository(seatPath);

validator.validateSeatFormat("12A");
validator.ensureSeatAvailable("AA101", "12A", seatRepo);
SeatReservation seat = SeatReservation.create("RES-1", "AA101", "PAX-1", "12A", "ECONOMY", 240.0);
seatRepo.save(seat);

TicketIssuanceRepository ticketRepo = new TicketIssuanceRepository(ticketPath);
BaggageRulesEngine baggageEngine = new BaggageRulesEngine();
TicketIssuance ticket = null;
try {
    double fee = baggageEngine.validateAndCalculateFee(40.0);
    ticket = TicketIssuance.issue("ETKT-1", "BKG-1", "Alice", "AA101", 40.0, fee);
    ticketRepo.save(ticket);
} catch (ExcessBaggageException ex) {
    seat.transitionTo(SeatReservationStatus.RELEASED);
    seatRepo.update(seat);
    throw ex;
}
```

After Facade: Single Immediate Friend

With the facade layer in place, the client interacts solely with `FlightBookingFacade`, passing a strongly-typed `BookingRequest` and receiving an atomic `BookingResult`:

```
FlightBookingFacade facade = getBookingFacade();
BookingRequest request = BookingRequest.builder()
    .flightNumber("AA101").passengerName("Alice").seatNumber("12A")
    .baggageWeightKg(18.0).memberId("MEM-101").mealPreference("VEGAN")
    .build();

BookingResult result = facade.bookFlight(request);
if (result.isSuccess()) {
    System.out.println("Flight Confirmed: " + result.getBookingReference());
} else {
    System.out.println("Booking Compensated: " + result.getFailureReason());
}
```

7. Written Reflection: Facade vs. Adapter

> **Architectural Intent Comparison:**
> The decision between Facade and Adapter must be argued strictly from architectural intent, not class count.

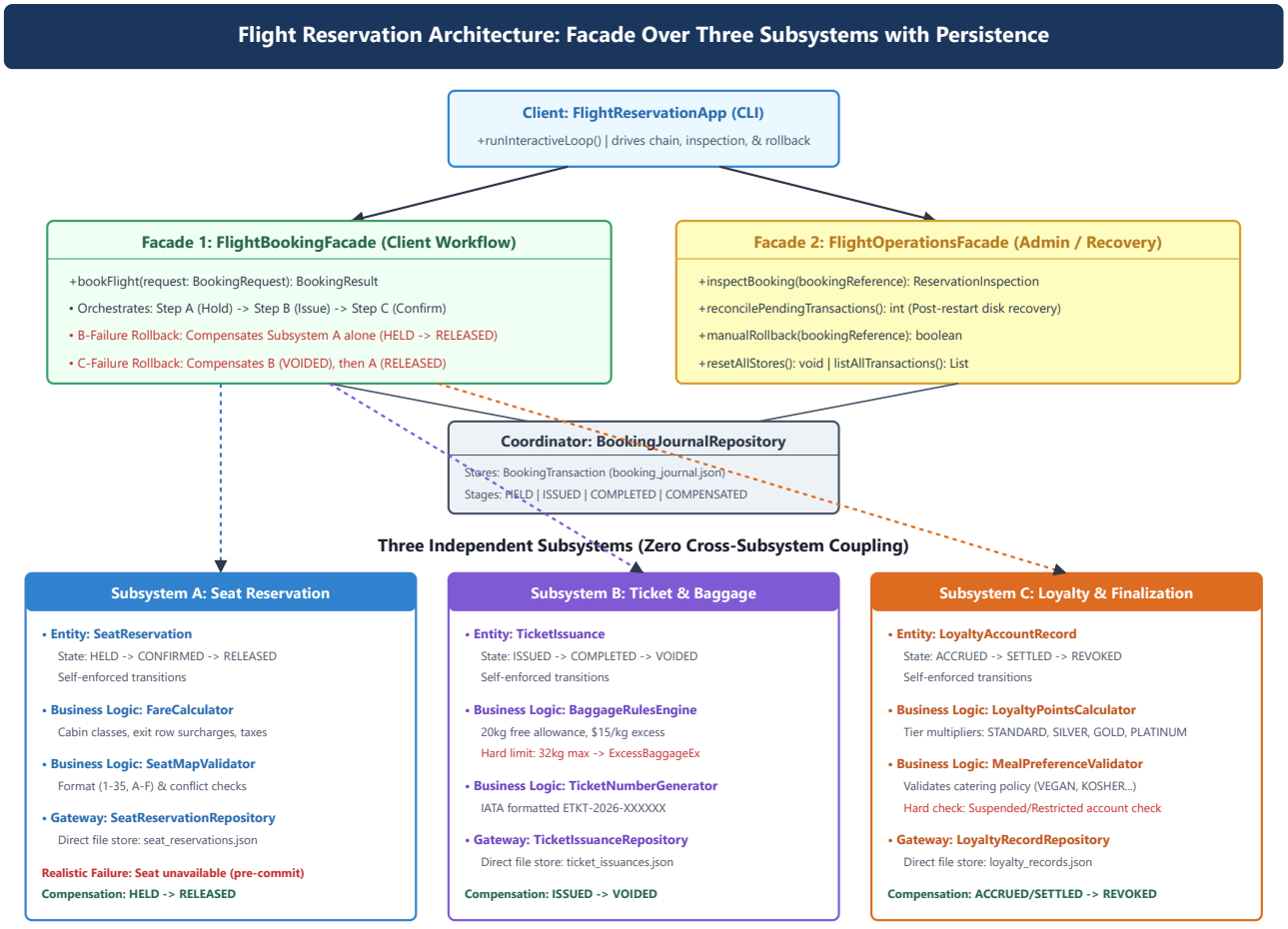
Adapter's Intent: Interface Translation. An Adapter wraps an existing, incompatible class to make it conform to an expected target interface. Its sole purpose is polymorphic substitutability: allowing a client to utilize a foreign or legacy component without modifying the client's code or the component's internal behavior.

Facade's Intent: Subsystem Encapsulation & Orchestration. A Facade provides a higher-level, simplified interface to an entire subsystem or cluster of subsystems. Rather than converting one interface into another, it introduces a brand new interface that encapsulates complex sequencing, multi-step coordination, transactional boundaries, and error recovery policies.

Why This Problem Calls for a Facade: In the Flight Reservation domain, Subsystems A, B, and C are not alternative implementations of a single concept; `SeatReservation` cannot and should not be adapted into a `TicketIssuance`. None of the subsystems are trying to adhere to a common interface. Instead, they represent distinct, cooperating capabilities that must execute in a strict sequential chain with cross-system rollback capabilities. The client does not seek interface compatibility; it requires an orchestrator that presents a single atomic booking operation while handling partial rollbacks behind the scenes. Therefore, Facade is the architecturally sound pattern, while Adapter is conceptually inapplicable.

8. Complete System UML Class Diagram

The vector diagram below illustrates the complete architecture, showing the Client (`FlightReservationApp`), the dual Facades (`FlightBookingFacade`, `FlightOperationsFacade`), the Coordinator journal, and all three decoupled subsystems with their associations:



9. Verification, Test DSL & Code Coverage

The codebase includes **36 unit tests** adhering to the F.I.R.S.T. testing principles (Fast, Independent, Repeatable, Self-Validating, Timely). Each test starts from a clean persisted state via JUnit 5 temporary directories (`@TempDir`).

Domain-Specific Testing Language (DSL)

A custom DSL helper (`FlightBookingDsl`) allows test scenarios to read as natural sentences in the flight domain:

```
FlightBookingDsl.given(bookingFacade, operationsFacade)
    .forPassenger("PAX-002", "Heavy Luggage Passenger")
    .flying("AA102")
    .requestingSeat("14B", "ECONOMY")
    .withBaggage(35.0)
    .withLoyalty("MEM-777", "STANDARD", "STANDARD")
    .whenBooking()
    .thenExpectFailureAtStepB()
    .andVerifyCompensatedAOnly();
```

JaCoCo Code Coverage Report

Package / Layer	Total Lines	Covered Lines	Line Coverage (%)	Status
`com.flightreservation.facade`	293	287	**98.0%**	Passed
`com.flightreservation.coordinator`	119	109	**91.6%**	Passed
`com.flightreservation.subsystem.seat`	137	126	**92.0%**	Passed
`com.flightreservation.subsystem.ticket`	109	100	**91.7%**	Passed
`com.flightreservation.subsystem.loyalty`	118	109	**92.4%**	Passed
`com.flightreservation.exception`	18	18	**100.0%**	Passed
TOTAL BUNDLE	**794**	**749**	**94.3%**	>80% Goal Exceeded

10. Live Interactive Demo & Commands

The system includes an interactive console application (`FlightReservationApp`) that allows the examiner to trigger the entire chain live, inspect disk stores, simulate application restarts, and verify rollbacks:

- **Run all unit tests:** `mvn clean test`
- **Launch live demo CLI:** `mvn compile exec:java`
- **Reset storage files:** Choose menu option `10` or delete contents of `data/`.